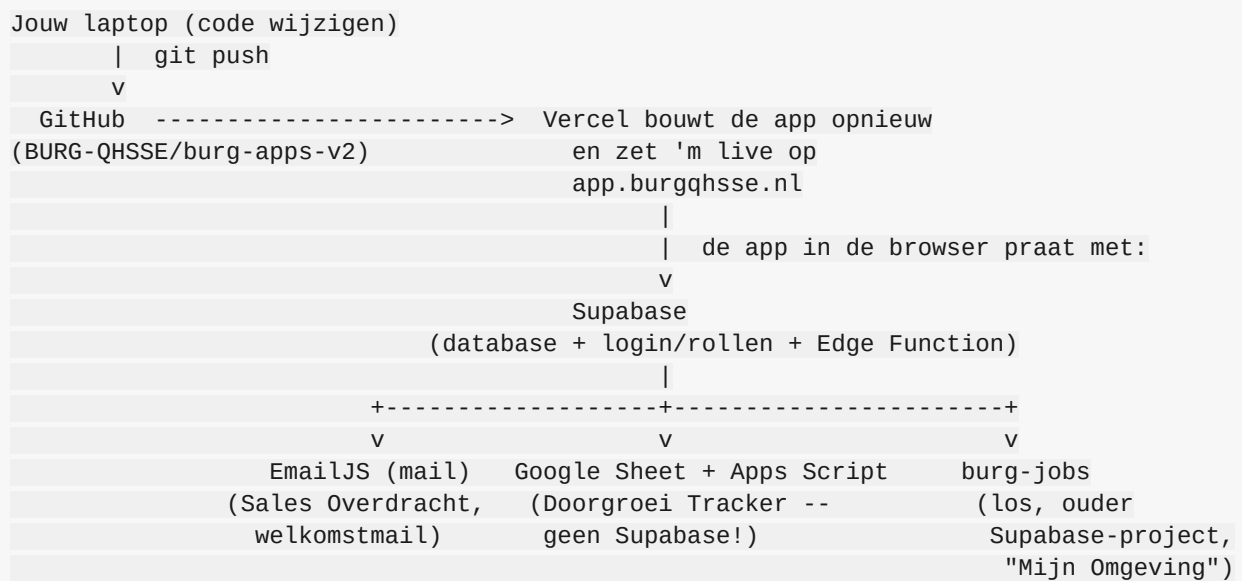


BURG Apps — Hoe de constructie in elkaar zit

Dit document legt de technische opzet uit: welke diensten er zijn, hoe ze op elkaar inspelen, en hoe een wijziging van "code op je laptop" naar "live voor iedereen" gaat. Bedoeld om iemand (bijv. Max) in één keer het hele plaatje te geven — voor de losse account-overdracht/eigenaarschap-details, zie `OVERDRACHT.md`/`OVERDRACHT.pdf`.

De keten in één plaatje



De vuistregel: GitHub is de bron van de code. Vercel bouwt en host die code. Supabase is waar de data en de logica achter inloggen/rollen leeft. De rest (EmailJS, Google Sheet, burg-jobs) zijn losse, aanvullende koppelingen die maar door één of twee specifieke tools gebruikt worden — niet de kern van de app.

1. GitHub — de broncode

Alle code staat in de repo `BURG-QHSSE/burg-apps-v2` (React + Vite). Dit is gewoon een Git-repository zoals je gewend bent: clonen, branchen, pull requests, mergen naar `main`.

Belangrijk: zodra iets naar de `main`-branch gepusht wordt, pikt Vercel dat automatisch op en bouwt/deployt de nieuwe versie — er is geen aparte "deploy-knop" die je moet indrukken voor gewone code-wijzigingen.

2. Vercel — hosting & automatische deploys

Vercel is puur de hosting-laag: het neemt de React-app uit GitHub, bouwt 'm (`npm run build`), en serveert het resultaat op `app.burgqhsse.nl`.

- Elke push naar `main` → automatische nieuwe **Production**-deploy.
- Elke push naar een andere branch / pull request → een aparte **Preview**-deploy met een eigen tijdelijke URL, handig om te testen voordat je merged.
- Vercel bewaart ook de **environment variables** (de Supabase-URL/sleutels die de app nodig heeft om te weten met welke database hij moet praten) — die staan dus niet in de code zelf, maar in Vercel's instellingen (Settings → Environment Variables).

Als de site niet update na een push: eerste check is altijd het **Deployments**-tabblad in Vercel — daar zie je of de build gefaald is en waarom.

3. Supabase — database, login en logica

Dit is het hart van de app. Eén Supabase-project ("BURG APPS V2", project-ref `kthriaekxqxijhqboxkd`) doet drie dingen:

a) Database

Alle tabellen staan gedefinieerd in `supabase/schema.sql` in de repo — dit bestand is bewust de complete, actuele bron van waarheid (elke tabel, kolom en functie die ooit is toegevoegd, staat hier met uitleg in comments). De belangrijkste tabellen: - `profiles` — alle gebruikers, hun rol (admin/manager/hr/user), of ze actief zijn, en yield-gerelateerde velden. - `role_audit_log` — wie heeft ooit wiens rol gewijzigd. - `tool_usage` — hoe vaak welke tool geopend wordt (voor de "App Counter" in het Adminpaneel). - `plaatsingen` — log van plaatsingen, voor de yield-berekening. - `proeftijd_kandidaten` — data voor de Proeftijd Tracker.

Let op: dit repo heeft géén migratie-tooling (geen Prisma/aparte migratie-map). Een wijziging aan de database gaat altijd in twee stappen: (1) pas `supabase/schema.sql` aan zodat het bestand up-to-date blijft, én (2) draai de daadwerkelijke SQL handmatig tegen de live database (via de Supabase SQL-editor in de browser, of `npx supabase db query --linked "..."` vanaf de command line).

b) Login & rollen (RLS)

Supabase Auth regelt het inloggen zelf. Wie wat mag zien in de database wordt afgedwongen via **Row Level Security (RLS)**-polities (ook in `schema.sql`) — bijvoorbeeld: een gewone gebruiker mag alleen zijn eigen profiel lezen, een admin mag alles lezen. Rolwijzigingen zelf lopen bewust nooit via een simpele database-update, maar altijd via een beveiligde functie (`change_user_role()`) die ook checkt dat je niet de laatste admin degradeert.

c) Edge Function — `admin-users`

Sommige acties (een nieuw account aanmaken met wachtwoord, een account permanent verwijderen) vereisen de Supabase `service_role`-sleutel — een "master key" die nooit in de browser mag komen.

Daarom draait dit in een **Edge Function** (server-side code, niet in de React-app zelf):
`supabase/functions/admin-users/index.ts`. Na een wijziging aan dit bestand moet je 'm opnieuw deployen: `npx supabase functions deploy admin-users`.

d) Twee verschillende Supabase-projecten — niet met elkaar verwarren!

- **BURG APPS V2** (`kthriaekxqxijhqboxkd`) — hierboven beschreven, de "hoofd"-database van deze app.
- **burg-jobs** (`ziwqshuabwcthqjspuso`) — een apart, ouder Supabase-project dat alleen gebruikt wordt door "Mijn Omgeving"/Kansen Swiper (vacatures, employees, swipe-data). Dit hoort bij het originele BURG-Apps-systeem waar Max al mee werkt. De twee databases weten niets van elkaar — ze staan naast elkaar, niet in elkaar.

4. EmailJS — mail versturen

Twee plekken in de app sturen e-mail: Sales Overdracht (vacature-overdracht naar een consultant) en de welkomstmail bij een nieuw account. Beide gebruiken dezelfde EmailJS-service (`service_tdpa3m9`) rechtstreeks vanuit de browser (JavaScript) — er is geen eigen mailserver, EmailJS is een extern mail-verstuur-dienstje waar de app een sleutel (public key, geen geheim) voor gebruikt.

5. Google Sheet + Apps Script — Doorgroei Tracker

Dit is de uitzondering op de regel: de Doorgroei Tracker praat **niet** met Supabase. In plaats daarvan is er een Google Sheet ("Doorgroei Tracker BURG") met een gekoppeld Apps Script-project. Dat script (`doGet()`-functie) leest de Sheet-tabbladen (Invoer, Invoer Sales, Weekdata Sales, Dashboard), rekent zelf de voortgang uit, en serveert het resultaat als JSON via een publieke URL. De React-app haalt die JSON gewoon op zoals bij elke andere API (`src/lib/doorgroeiTrackerApi.js`).

Waarom zo raar? Omdat de Doorgroei Tracker oorspronkelijk als losstaand Sheet-hulpmiddel is gebouwd, en het makkelijker was om de bestaande Sheet te blijven gebruiken dan alle historische data over te zetten naar Supabase.

6. Lokaal ontwikkelen — hoe je zelf wijzigingen maakt en live zet

1. Clone de repo, `npm install`.
2. Kopieer `.env.example` naar `.env`, vul 'm met de Supabase-projectgegevens (URL + anon key — deze sleutel is publiek/client-side, beveiliging loopt via RLS, niet via geheimhouding).
3. `npm run dev` — lokale ontwikkelserver.
4. Wijzigingen maken, `npm run lint` + `npm run build` checken dat alles nog klopt.
5. Commit + push naar `main` (of via een pull request) → Vercel bouwt en deployt automatisch.

6. Database-wijziging? Vergeet niet ook de SQL handmatig tegen de live database te draaien (zie hierboven) — dat gebeurt niet automatisch bij een push.

Snel overzicht: waar zoek ik wat?

Vraag	Waar kijken
"Is de laatste versie live?"	Vercel → Deployments-tabblad
"Wat zit er precies in de database?"	<code>supabase/schema.sql</code> in de repo
"Waarom werkt inloggen niet?"	Supabase → Authentication → Users, en → URL Configuration (Site URL/Redirect URLs)
"Waarom komt een mail niet aan?"	EmailJS-dashboard → History/Logs
"Waarom klopt de Doorgroei Tracker niet?"	De Google Sheet zelf, of Uitbreidingen → Apps Script → Uitvoeringslogboek
"Wie mag wat?"	<code>src/lib/toolRegistry.js</code> (tools) en RLS-polities in <code>supabase/schema.sql</code> (data)